

Edutech Solution

Full Stack Development Internship — Task 5

Topic	State Management & Component Lifecycle
Tools	React Hooks (useState, useEffect)
Dataset	Todo Logic
Submitted by	Intern
Date	2026-06-01

1. Introduction

This report documents the design, development, and outcomes of Task 5 of the Edutech Solution Full Stack Development Internship. The objective was to build a stateful Todo application using React Hooks — specifically **useState** and **useEffect** — demonstrating a solid understanding of component lifecycle, state management, and data flow across components.

2. Objective

- Implement a fully functional Todo application in React.
- Use **useState** to manage component-level state (todo list, input, filter).
- Use **useEffect** to handle side effects (localStorage sync, document title).
- Demonstrate **prop drilling** by passing state and handlers across components.
- Apply filter logic (All / Active / Completed) to manage complex data flows.

3. Tools & Technologies

Technology	Version / Notes	Purpose
React	18+	UI library
useState	Built-in Hook	Local state management
useEffect	Built-in Hook	Side-effects & lifecycle
localStorage	Web API	Persistent storage
Vite / CRA	Build tool	Project scaffolding

4. Implementation

4.1 Component Architecture

The application is split into three components to follow good separation-of-concerns principles:

Component	Role
TodoApp	Root component — owns all state (todos, inputText, filter)
TodoItem	Renders a single todo; receives todo object + handlers via props
FilterBar	Renders filter buttons; receives filter & setFilter via props (prop drilling)

4.2 State Management with useState

Three state variables are maintained at the root level:

```
const [todos, setTodos] = useState([]); const [inputText, setInputText] =  
  useState(''); const [filter, setFilter] = useState('all');
```

todos is initialised lazily from localStorage so data survives page refreshes. Each todo is an object with *id*, *text*, and *completed* fields.

4.3 Side Effects with useEffect

Two useEffect hooks handle side effects:

```
// 1. Persist todos to localStorage useEffect(() => { localStorage.setItem('todos',  
  JSON.stringify(todos)); }, [todos]); // 2. Update browser tab title useEffect(() => {  
  const pending = todos.filter(t => !t.completed).length; document.title = pending > 0 ?  
    `${pending} Todo App` : 'Todo App'; }, [todos]);
```

Both effects declare **[todos]** as their dependency array, ensuring they run only when the todo list changes — not on every render.

4.4 Key Features

- **Add task** — via button click or Enter key press.
- **Toggle completion** — checkbox toggles the completed flag.
- **Delete task** — removes a task from state.
- **Filter (All / Active / Completed)** — derived from current state.
- **Clear completed** — bulk-removes finished tasks.
- **Persistence** — todos survive browser refresh via localStorage.
- **Live tab title** — shows pending task count as a badge.

5. Interview Questions & Answers

Q1: What is the purpose of useEffect?

useEffect is a React Hook that lets you perform side effects in functional components. Side effects include data fetching, subscriptions, manually changing the DOM, or interacting with browser APIs such as localStorage. It accepts a callback and an optional dependency array. When the dependency array is provided, the effect runs only when one of the listed values changes. An empty array [] means the effect runs once after the initial render (similar to componentDidMount). The hook can also return a cleanup function, which React calls before re-running the effect or when the component unmounts.

Q2: Explain Prop Drilling.

Prop drilling refers to the practice of passing data from a parent component down through multiple layers of child components via props, even if the intermediate components do not use that data themselves. In this Todo app, the filter state and setFilter function are owned by TodoApp and passed down to FilterBar — a classic two-level drill. While manageable for shallow trees, prop drilling becomes problematic in deeply nested hierarchies. Common solutions include the React Context API, state management libraries (Redux, Zustand), or component composition patterns.

6. Outcome & Learning

By completing this task the intern successfully:

- ✓ Built a production-ready stateful React application from scratch.
- ✓ Applied useState to manage multiple pieces of UI state cleanly.
- ✓ Applied useEffect to synchronise state with browser APIs (localStorage, document.title).
- ✓ Understood the concept and practical implications of prop drilling.
- ✓ Experienced derived state patterns (filtered lists, counts) without redundant state.
- ✓ Practiced component decomposition and single-responsibility design.